Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- **Web API Service with Basic Authentication**
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

Web API Service with Basic Authentication

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

Web API Service with Basic Authentication

In this article, I am going to discuss **how to consume Web API Services with Basic Authentication**. Please read our previous article before proceeding to this article, where we discussed how to implement the **Role-Based Basic Authentication in Web API** with an example. We are also going to work with our previous example. As part of this article, we are going to discuss the following points:

1. **How to enable CORS in Web API?**
2. **How to call Web API service from JQuery using Ajax?**
3. **How to call Web API service from C# using a Console Application?**

How to Enable CORS in Web API?

As we are going to consume the Web API Service using Jquery Ajax from another domain, we need to enable CORS in our application. Enabling CORS is a two steps process.

Step1: Install **Microsoft.AspNet.WebApi.Cors** package. Execute the following command using the NuGet Package Manager Console.

Step2: Include the following 2 lines of code in **Register()** method of **WebApiConfig** class in **WebApiConfig.cs** file which is inside **App_Start** folder.

```
EnableCorsAttribute cors = new EnableCorsAttribute("*", "*", "*");
config.EnableCors();
```

With the above two lines of code in place, the WebApiConfig class should look as below.

```
using System.Web.Http;
using System.Web.Http.Cors;
namespace RoleBasedBasicAuthentication.App_Start
{
    public static class WebApiConfig
    {
        public static void Register(HttpConfiguration config)
        {
            // Web API routes
            config.MapHttpAttributeRoutes();

            config.Routes.MapHttpRoute(
                name: "DefaultApi",
                routeTemplate: "api/{controller}/{id}",
                defaults: new { id = RouteParameter.Optional }
            );

            EnableCorsAttribute cors = new EnableCorsAttribute("*", "*", "*");
            config.EnableCors();
        }
    }
}
```

If you are new to CORS, then I strongly recommended you to read the following articles, where I discussed CORS in details.

https://dotnettutorials.net/lesson/cross-origin-resource-sharing-web-api/

Modifying the Controller:

In the controller, we need to create one method called GetEmployees. Depending on the credentials provided the Web API service should authenticate and return the correct results as follows.

If the AdminUser username and password are provided then only the male employees are returned from the service. Similarly, if SuperadminUser username and password are provided then only the female employees are returned from the service. In the same way, if both user username and password are provided then all the employees should be returned from the service else any other case it should return BAD Request from the service

To achieve this, let's modify the Employee Controller as shown below.

```
using RoleBasedBasicAuthentication.Models;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Security.Claims;
using System.Threading;
using System.Web.Http;

namespace RoleBasedBasicAuthentication.BASIC.Controllers
{
    public class EmployeeController : ApiController
    {
        [BasicAuthentication]
        [AuthorizationFilter("*", "*", "*")]
        [MyAuthorize(Roles = "Admin,SuperAdmin")]
        [Route("api/Employees")]
        public HttpResponseMessage GetEmployees()
        {
            // var identity = (ClaimsIdentity)User.Identity;
            //var username = identity.Name;
            //OR you can use the below code to get the login username
            string username = Thread.CurrentPrincipal.Identity.Name;

            var EmpList = new EmployeeBL().GetEmployees();
            switch (username.ToLower())
            {
                case "admin user":
                    return Request.CreateResponse(HttpStatusCode.OK,
                        EmpList.Where(e => e.Gender.ToLower() == "male").ToList());
                case "superadminuser":
                    return Request.CreateResponse(HttpStatusCode.OK,
                        EmpList.Where(e => e.Gender.ToLower() == "female").ToList());
                case "bothuser":
                    return Request.CreateResponse(HttpStatusCode.OK,EmpList);
                default:
                    return Request.CreateResponse(HttpStatusCode.BadRequest);
                }
            }
        }
    }
```

In the last article, we discussed how to consume the Web APIs using Postman as a client. But it is also necessary to know how to consume APIs from different types of clients.

Consuming Web API Service using JQuery AJAX:

Create one empty web application and then add one HTML page with the name AJAXClient to your application. Once you add the HTML Page then copy and paste the following code.

Note: You need to be installed JQuery packages in your projects.

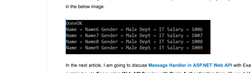
```
<!DOCTYPE html>
<html>
<head>
    <title></title>
    <meta charset="utf-8" />
    <!-- First Installed jquery into your application-->
    <script src="Scripts/jquery-3.3.1.js"></script>
    <script type="text/javascript">
        $(document).ready(function () {
            var uiEmployees = $('#uiEmployees');

            $('#btnGetData').click(function () {
                // Get the username & password from textboxes
                var username = $('#txtUsername').val();
                var password = $('#txtPassword').val();

                $.ajax({
                    type: 'GET',
                    // Make sure to change the port number to
                    // where you have the service
                    // running on your local machine
                    url: 'http://localhost:61312/api/Employees',
                    dataType: 'json',
                    // Specify the authentication header
                    // btoa() method encodes a string to Base64
                    headers: {
                        'Authorization': 'Basic ' + btoa(username + ':' + password)
                    },
                    success: function (data) {
                        uiEmployees.empty();
                        $.each(data, function (index, val) {
                            var EmployeeDetails = 'Name = ' + val.Name + ' Gender = ' +
val.Gender + ' Dept = ' + val.Dept + ' Salary = ' + val.Salary;
                            uiEmployees.append('<li> ' + EmployeeDetails + '</li>')
                        });
                    },
                    complete: function (jqXHR) {
                        if (jqXHR.status == '401') {
                            uiEmployees.empty();
                            uiEmployees.append('<li style="color:red">'
                                + jqXHR.status + ' : ' + jqXHR.statusText + '</li>')
                        }
                    }
                });
            });

            $('#btnClear').click(function () {
                uiEmployees.empty();
            });
        });
    </script>
</head>
<body>
    Username : <input type="text" id="txtUsername" />
    Password : <input type="password" id="txtPassword" />
    <br /><br />
    <input id="btnGetData" type="button" value="Get Employees" />
    <input id="btnClear" type="button" value="Clear" />
    <ul id="uiEmployees"></ul>
</body>
</html>
```

First, run the service application and then the client application. Now go to the client application and navigate to the URL **/AJAXClient.html**. Provide the valid username and password and click on the GetEmployees button and you should see the data as expected.



Now provide the wrong username and password and see the results as 401 unauthorized as expected.

Consuming Web API Service using C# Console Application:

Create a console application and then copy and paste the following code. You need to Install **Newtonsoft.Json** Package into your application

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Net.Http.Headers;
using Newtonsoft.Json;

namespace ClientAppLication
{
    class Program
    {
        static void Main(string[] args)
        {
            HttpClientHandler handler = new HttpClientHandler();
            HttpClient client = new HttpClient(handler);

            client.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Authorization",
Convert.ToBase64String(Encoding.Default.GetBytes("Administrator:123456")));

            //Need to change the PORT number where your WEB API service is running
            var result = client.GetAsync(new
Uri("http://localhost:61312/api/Employees").Result;

            if (result.IsSuccessStatusCode)
            {
                Console.WriteLine("Done" + result.StatusCode);
                var JsonContent = result.Content.ReadAsStringAsync().Result;

                List<Employee> empList = JsonConvert.DeserializeObject<List<Employee>>
(JsonContent);

                foreach(var emp in empList)
                {
                    Console.WriteLine("Name = " + emp.Name + " Gender = " + emp.Gender +
" Dept = " + emp.Dept + " Salary = " + emp.Salary);
                }
            }
            else
            {
                Console.WriteLine("Error" + result.StatusCode);
                Console.ReadLine();
            }
        }

    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string Dept { get; set; }
        public int Salary { get; set; }
    }
}
```

Here, we have given the AdminUser username and password, so it displays only the male employees as expected as shown in the below image.

```
Done200
Name = Name6 Gender = Male Dept = IT Salary = 1006
Name = Name7 Gender = Male Dept = IT Salary = 1007
Name = Name8 Gender = Male Dept = IT Salary = 1008
Name = Name9 Gender = Male Dept = IT Salary = 1009
```

In the next article, I am going to discuss **Message Handler in ASP.NET Web API** with Examples. Here, in this article, I try to explain how to **Consuming Web API Service with Basic Authentication** from JQuery AJAX as well as from a C# client.

Dot Net Tutorials
About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 12-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

4 thoughts on “Web API Service with Basic Authentication”

RAMPRASAD MONDAL
DECEMBER 12, 2019 AT 8:45 PM

Brilliant !

Reply

ABHI
DECEMBER 31, 2019 AT 2:29 PM

What if instead cg get Post is there??

Reply

ANDREW HEGAI
DECEMBER 20, 2020 AT 4:57 PM

Im not familiar with ajax and for some reason those buttons in AJAXClient.html *page do nothing*. I have JQuery package in project. Maybee missing something to make ajax run?

Reply

ANDREW HEGAI
DECEMBER 20, 2020 AT 4:46 PM

AJAXClient.html page not work because of this string
Scripts/jquery-3.3.1.js
need to update jquery version to 3.4.1
Scripts/jquery-3.4.1.js

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel